

Acheter plusieurs abonnements — guide mobile

Un utilisateur paie N abonnements d'un coup et reçoit N codes à usage unique, qu'il distribue à qui il veut : une école dote ses élèves, un parent équipe ses enfants, une entreprise ses employés.

Les réponses ci-dessous sont issues d'**appels réels** sur l'environnement de développement.

Référence : issue [#247](#). Complète le [guide des abonnements](#) et celui des [codes de réduction](#).

1. Le parcours en trois temps

1. POST /codes/commandes → la commande, et le montant à payer
2. POST /paiements/initier → l'URL de paiement (mécanique habituelle)
3. le webhook confirme → les codes naissent et partent par courriel

Rien n'est créé avant le paiement. Une commande abandonnée ne laisse aucun code derrière elle — inutile de la nettoyer, inutile de la craindre.

L'écran d'achat n'a donc que deux champs : le **plan** et la **quantité**.

2. Créer la commande

```
POST /codes/commandes
Authorization: Bearer <token>

{ "plan_uuid": "...", "quantite": 3 }
```

```
{
  "uuid": "711b3735-1885-4ae3-895f-567f1d3ffe65",
  "quantite": 3,
  "prix_unitaire": 18000,
  "montant_total": 54000,
  "devise": "XOF",
  "statut": "EN_ATTENTE",
  "plan": { "code": "ANNUEL", "libelle": "Abonnement annuel" }
}
```

Affichez `montant_total` tel qu'il est renvoyé. Ne recalculez pas `quantité × prix` côté mobile : le prix est **figé à la commande**, et si le tarif du plan change entre-temps, votre calcul divergerait de ce qui sera réellement débité.

contrainte	valeur
quantité minimale	1
quantité maximale	500
plan	doit être ouvert à la vente

Au-delà de 500, l'API répond `400` avec un message à afficher tel quel : c'est une commande sur mesure, à traiter hors application.

3. Payer

Exactement la même mécanique qu'un abonnement, **avec un champ différent** :

```
POST /paiements/initier

{ "commande_uuid": "711b3735-...", "prestataire": "FEDAPAY" }
```

⚠ `commande_uuid` **remplace** `abonnement_uuid`. Les deux sont exclusifs : un appel qui porterait les deux est rejeté en `400`. N'envoyez que celui qui correspond à ce que l'utilisateur achète.

La réponse est celle que vous connaissez — `url_paiement`, `token_client` — et la suite ne change pas : redirection, retour, attente du webhook.

L'URL de retour pointe vers `/mes-codes` au lieu de `/abonnements`.

4. Suivre ses commandes

GET `/codes/commandes`

```
[{
  "uuid": "711b3735-...",
  "statut": "PAYEE",
  "quantite": 3,
  "prix_unitaire": 18000,
  "montant_total": 54000,
  "devise": "XOF",
  "plan": { "code": "ANNUEL", "libelle": "Abonnement annuel" },
  "date_creation": "2026-09-23T21:23:23.322Z",
  "date_paieement": "2026-09-23T21:29:10.004Z",
  "codes_generees": 3
}]
```

statut	ce que voit l'utilisateur
EN_ATTENTE	paiement non abouti — proposez de reprendre
PAYEE	livrée, <code>codes_generees</code> en donne le nombre
ANNULEE / REMBOURSEE	close

`codes_generees` vaut 0 tant que la commande n'est pas payée. Ce n'est pas un échec, c'est l'état normal avant paiement.

5. La liste des codes — l'écran principal

GET `/codes/mes-codes`

```
[
  {
    "code": "EDK-FUEYKK2R",
    "origine": "ACHAT",
    "libelle": null,
    "est_actif": true,
    "utilise": false,
    "utilise_le": null,
    "utilise_par": null,
    "usages_restants": 1,
    "date_creation": "2026-09-23T21:29:10.121Z"
  },
  {
    "code": "EDK-YG5AEYLZ",
    "origine": "ACHAT",
    "utilise": true,
    "utilise_le": "2026-09-24T08:14:02.339Z",
    "utilise_par": { "nom": "Awa Diop", "email": "awa@example.com" },
    "usages_restants": 0
  },
  {
    "code": "HB0964",
    "origine": "INSCRIPTION",
    "utilise": false,
    "usages_restants": null
  }
]
```

Ce qu'il faut comprendre de cette réponse

La liste mélange deux natures de codes. `origine` les distingue :

origine	ce que c'est	usage
ACHAT	un code acheté, à distribuer	une seule fois
INSCRIPTION	son code de parrainage personnel	sans limite
ADMIN	un code reçu d'une campagne	selon le cas

Si l'écran est « mes codes achetés », **filtrez sur** `origine === "ACHAT"` — sinon le code de parrainage s'y retrouvera, avec un comportement différent.

`usages_restants: null` signifie « sans limite », pas « zéro ». Un client qui afficherait `null` comme `0` montrerait un code de parrainage épuisé alors qu'il est illimité.

utilise_par est le **bénéficiaire, pas l'acheteur**. C'est l'information que cherche celui qui a distribué ses codes : qui s'en est servi. Ses champs peuvent être `null` si le compte a depuis été supprimé.

Testez `utilise`, pas `usages_restants === 0`. Les deux coïncident pour un code acheté, mais pas pour un code illimité, où `usages_restants` vaut toujours `null`.

6. Utiliser un code

Rien de nouveau : c'est le champ `code` de la souscription habituelle.

```
POST /abonnements/souscrire
{ "plan_uuid": "...", "code": "EDK-FUEYKK2R" }
```

Le code ouvre l'abonnement **sans paiement** — c'est déjà payé par l'acheteur.

L'acheteur peut utiliser un de ses propres codes

C'est une exception délibérée : il l'a payé, rien ne l'oblige à le donner. Un parent qui équipe sa famille se compte dedans.

Elle ne vaut **que pour les codes achetés**. Un code de parrainage reste refusé à son propriétaire, avec le motif `AUTO_UTILISATION` — s'auto-parrainer n'a aucun sens.

Vérifier avant de souscrire

```
POST /codes/valider
{ "code": "EDK-FUEYKK2R", "plan_uuid": "..." }
```

```
{ "valide": true, "effets": { "abonnement_offert": { "duree_jours": null } } }
```

Cet endpoint **ne demande pas de compte** : un code peut être vérifié avant l'inscription. Envoyez tout de même le jeton quand l'utilisateur est connecté — sans lui, deux refus ne peuvent pas être détectés (`DEJA_UTILISE` et `AUTO_UTILISATION`), et l'aperçu serait optimiste.

7. Le courriel

Les codes partent automatiquement à l'adresse de l'acheteur dès le paiement confirmé. **Ne comptez pas dessus comme unique canal** : si le serveur de messagerie est indisponible, la commande reste honorée et les codes restent accessibles par `GET /codes/mes-codes`.

C'est même la raison d'être de cet écran. Prévoyez-y une action **copier** sur chaque code, et idéalement un partage — c'est ainsi que l'acheteur les distribuera en pratique.

8. Ce qu'il y a à faire

1. Écran d'achat : choix du plan, quantité (1 à 500), **montant renvoyé par l'API** et non recalculé.

1. Paiement : `POST /paiements/initier` avec `commande_uuid`, jamais les deux champs à la fois.

1. Écran « mes codes » : filtrer sur `origine === "ACHAT"`, traiter

`usages_restants: null` comme « illimité », afficher `utilise_par` quand il existe.

1. Action **copier** sur chaque code — c'est le geste principal de cet écran.
2. Écran « mes commandes » : reprendre un paiement resté `EN_ATTENTE` .
3. Ne pas coder en dur le plafond de 500 ni les prix : l'API fait foi.

9. Points de vigilance

Le montant vient du serveur. Le prix est figé à la commande ; un calcul local divergerait si le tarif change.

Un code acheté n'expire pas. Il reste valable tant qu'il n'a pas servi — n'affichez pas de date d'expiration, il n'y en a pas.

La commande ne crée rien avant paiement. Un utilisateur qui abandonne n'a aucun code, et sa commande reste `EN_ATTENTE` — proposez-lui de reprendre plutôt que de recommencer.